TECHNICAL RESEARCH REPORT

Rule-Based Expert System for Medical Diagnosis

Fever-Related Disease Identification Using Forward-Chaining Inference

Artificial Intelligence | B.Tech Computer Science | ASET | March 2026

Report Type	Open-Ended Experiment — Technical Implementation Report
Subject	Artificial Intelligence
Branch / Programme	B.Tech Computer Science Engineering
Institution	ASET (Amity School of Engineering & Technology)
Authors	Kislay Dubey, Aryan Bhargav & Abhinav Sharma
Date of Submission	March 2026
Programming Language	Python 3.x (Standard Library Only)
Experiment Category	Rule-Based AI / Expert Systems / Medical Decision Support

1. Executive Summary

This report documents the design, implementation, and validation of a Rule-Based Expert System (RBES) that emulates a clinical decision-support assistant for the diagnosis of fever-related diseases. The system was implemented entirely in Python 3.x without any external libraries, relying solely on native conditional logic, list structures, and functions to realise a transparent inference engine.

The knowledge base encodes seven diagnostic rules covering Influenza, Malaria, Dengue Fever, COVID-19, Typhoid, Strep Throat, and Common Cold. Given a patient's symptoms, body temperature, travel history, and duration of illness, the system applies Forward Chaining to produce a ranked Differential Diagnosis list—each entry annotated with a confidence score, risk level, and recommended clinical action.

Three representative patient test cases were executed and verified against expected diagnoses, confirming the system's correct rule-firing behaviour. The experiment illustrates core Artificial Intelligence concepts including knowledge representation, inference mechanisms, and confidence-based reasoning, within a medically meaningful application context.

2. Introduction

Artificial Intelligence has long held promise in the field of clinical medicine. Among the earliest and most practically deployed AI techniques are Expert Systems—programs that capture the reasoning of domain specialists and apply it algorithmically to novel problems. In healthcare, such systems can assist

practitioners in rapid preliminary diagnosis, particularly in resource-limited environments or during high-volume patient intake.

Fever is one of the most common presenting symptoms across a wide spectrum of diseases, many of which share overlapping symptom profiles. Distinguishing between Influenza, Malaria, Dengue, COVID-19, Typhoid, Strep Throat, and the Common Cold requires careful synthesis of multiple factors: the magnitude of fever, the specific constellation of accompanying symptoms, exposure or travel history, and the duration of illness.

This experiment constructs a miniature expert system that codifies this reasoning as a set of explicit IF-THEN production rules. The goals are threefold: (i) to demonstrate Rule-Based AI in a concrete, applied context; (ii) to implement a functioning forward-chaining inference engine in pure Python; and (iii) to produce clinically meaningful outputs—ranked differential diagnoses with actionable recommendations—from structured patient input.

3. Objectives

The experiment was undertaken with the following specific objectives:

- To design and implement a Rule-Based Expert System in Python that simulates a medical diagnosis assistant for fever-related diseases.
- To encode domain knowledge as seven IF-THEN production rules covering Influenza, Malaria, Dengue Fever, COVID-19, Typhoid, Strep Throat, and Common Cold.
- To implement a Forward Chaining inference engine that matches patient facts against stored rules and derives diagnoses.
- To assign confidence scores and risk levels to each diagnosis, enabling ranked differential diagnosis output consistent with clinical practice.
- To validate system correctness using three representative test patient cases and verify expected rule-firing behaviour.
- To produce a transparent, traceable AI system that requires no external libraries, making the underlying logic fully inspectable.

4. Theoretical Background

4.1 Expert Systems in Artificial Intelligence

An Expert System (ES) is a class of AI program designed to replicate the decision-making capability of a human specialist within a narrowly defined domain. Unlike general-purpose AI, expert systems operate within a bounded problem space using explicit, human-authored knowledge—rather than learned statistical patterns—making their reasoning fully traceable and auditable.

Expert systems were among the first commercially successful AI deployments, with MYCIN (1970s, Stanford University) being a landmark example in medical diagnosis for bacterial infections. They remain relevant today as decision-support tools in medicine, engineering, finance, and law.

4.2 Architecture: Three Core Components

A Rule-Based Expert System comprises three principal components:

(a) Knowledge Base

The Knowledge Base stores domain expertise as a structured collection of IF-THEN production rules. Each rule defines a set of conditions (the antecedent) and a corresponding conclusion (the consequent). In this system, conditions include fever temperature thresholds, specific symptom flags, travel history flags, and duration of illness; conclusions are diagnostic labels paired with confidence scores, risk levels, and recommended clinical actions.

(b) Working Memory (Fact Base)

The Working Memory holds the current state of the problem being solved—in this case, the patient's reported data: symptoms as a list of string tokens, body temperature as a float, travel history as an enumerated string, and duration in days as an integer. As the inference engine fires rules, new facts (diagnoses) are asserted into working memory.

(c) Inference Engine

The Inference Engine is the reasoning mechanism that links the Knowledge Base with Working Memory. It iterates through all rules, evaluates whether each rule's conditions are satisfied by the current facts, and—where they are—asserts the corresponding conclusion. In this implementation, all applicable rules are evaluated in a single pass (breadth-first), and results are subsequently ranked by confidence score.

4.3 Forward Chaining

This system employs Forward Chaining (also called data-driven reasoning), which begins with known facts and applies rules progressively to derive new conclusions. The algorithm proceeds as follows:

- Start with the initial fact set (patient symptoms, temperature, history, duration).
- Evaluate the IF-part of each rule against the current fact set.
- For every rule whose conditions are fully satisfied, assert the THEN-part as a new fact (a candidate diagnosis).
- Continue until all rules have been evaluated.
- Return the complete set of matched diagnoses, ranked in descending order of confidence.

Forward Chaining is well-suited to diagnostic tasks where all patient data is collected upfront and the goal is to derive all applicable conclusions—mirroring the clinical practice of constructing a differential diagnosis list rather than seeking a single definitive answer at the outset.

4.4 Confidence Scoring

Each production rule carries a confidence score in the range 0–100%, representing the strength of association between the specified symptom pattern and the corresponding disease. Scores were assigned based on clinical reasoning about symptom specificity and sensitivity:

Disease	Confidence	Risk Level	Key Discriminating Factors
Influenza (Flu)	90%	High	High fever + dry cough + body aches
Dengue Fever	88%	High	Very high fever + headache + joint pain + rash
Malaria	85%	High	Fever + endemic travel + chills + shivering
COVID-19	82%	Medium-High	Dry cough + loss of taste + mild fever
Typhoid	80%	Medium	Prolonged fever ≥39°C + abdominal pain + weakness
Strep Throat	78%	Medium	Fever + sore throat + swollen nodes (no body aches)

Common Cold	75%	Low	Low fever + runny nose + sneezing + sore throat

5. System Design & Implementation

5.1 Language & Environment

The system was implemented in Python 3.x using only the standard library. This deliberate constraint ensures that the rule-matching logic is fully transparent—there are no hidden statistical models, external knowledge graphs, or pre-trained components. The code can be executed in any Python-compatible environment: VS Code, PyCharm, IDLE, Google Colab, Jupyter Notebook, or the command-line terminal.

5.2 Input Parameters

Parameter	Data Type	Description
`patient_name`	String	Identifier for the patient case (display only)
`symptoms`	List[String]	Token list of presenting symptoms (e.g., dry_cough, chills, rash)
`fever_temp`	Float (°C)	Measured body temperature in degrees Celsius
`travel_history`	String	"endemic_area" if recent travel to disease-endemic region; "none" otherwise
`duration_days`	Integer	Number of days since symptom onset

5.3 Knowledge Base — Production Rules

The seven production rules encoded in the knowledge base are summarised in the table below:

#	Disease	Condition (IF-part)	Conf.	Action (THEN-part)
R1	**Influenza**	fever > 38.5°C AND dry_cough AND body_aches	**90%**	Rest, fluids, antivirals
R2	**Malaria**	fever ≥ 38.0°C AND endemic_area AND chills AND shivering	**85%**	Urgent blood smear test
R3	**Dengue**	fever > 39.0°C AND severe_headache AND joint_pain AND rash	**88%**	Platelet count; hospitalization
R4	**COVID-19**	dry_cough AND loss_of_taste AND fever ≥ 37.5°C	**82%**	RT-PCR; isolation; O_2 monitor
R5	**Typhoid**	fever ≥ 39.0°C AND duration ≥ 5 days AND abdominal_pain AND weakness	**80%**	Widal test; antibiotics
R6	**Strep Throat**	fever ≥ 38.0°C AND sore_throat AND swollen_lymph_nodes AND NOT body_aches	**78%**	Throat swab; antibiotics
R7	**Common Cold**	fever < 38.5°C AND runny_nose AND sneezing AND sore_throat	**75%**	Rest, fluids; symptomatic

5.4 Program Code

The full source code is presented below. The diagnose() function implements the Knowledge Base as a series of IF-THEN conditionals. The run_diagnosis() function acts as the controller—it calls diagnose(), sorts results by confidence, and prints a formatted report.

```
# Rule-Based Expert System for Medical Diagnosis (Fever-Related Diseases)
# Knowledge Base: IF-THEN Rules

def diagnose(symptoms, fever_temp, travel_history, duration_days):
    results = []

    # Rule 1: Influenza
    if fever_temp > 38.5 and "dry_cough" in symptoms and "body_aches" in symptoms:
        results.append(("Influenza (Flu)", 90, "High", "Rest, fluids, antiviral
medication"))

    # Rule 2: Malaria
    if fever_temp >= 38.0 and travel_history == "endemic_area" \
            and "chills" in symptoms and "shivering" in symptoms:
        results.append(("Malaria", 85, "High", "Urgent Blood Smear Test Required"))

    # Rule 3: Dengue Fever
    if fever_temp > 39.0 and "severe_headache" in symptoms \
            and "joint_pain" in symptoms and "rash" in symptoms:
        results.append(("Dengue Fever", 88, "High", "Platelet count test; hospitalization
may be needed"))

    # Rule 4: COVID-19
    if "dry_cough" in symptoms and "loss_of_taste" in symptoms and fever_temp >= 37.5:
        results.append(("COVID-19", 82, "Medium-High", "RT-PCR test, isolation, monitor
oxygen levels"))

    # Rule 5: Typhoid
    if fever_temp >= 39.0 and duration_days >= 5 \
            and "abdominal_pain" in symptoms and "weakness" in symptoms:
        results.append(("Typhoid", 80, "Medium", "Widal test, antibiotics course"))

    # Rule 6: Strep Throat
    if fever_temp >= 38.0 and "sore_throat" in symptoms \
            and "swollen_lymph_nodes" in symptoms and "body_aches" not in symptoms:
        results.append(("Strep Throat", 78, "Medium", "Throat swab culture,
antibiotics"))

    # Rule 7: Common Cold
    if fever_temp < 38.5 and "runny_nose" in symptoms \
            and "sneezing" in symptoms and "sore_throat" in symptoms:
        results.append(("Common Cold", 75, "Low", "Rest and fluids, symptomatic
treatment"))

    return results

def run_diagnosis(patient_name, symptoms, fever_temp, travel_history, duration_days):
    print("=" * 60)
    print("  MEDICAL AI EXPERT SYSTEM - FEVER DIAGNOSIS REPORT")
    print("=" * 60)
    print(f"  Patient Name  : {patient_name}")
    print(f"  Fever Temp    : {fever_temp} degC")
    print(f"  Symptoms      : {', '.join(symptoms)}")
    print(f"  Travel History: {travel_history}")
    print(f"  Illness Since : {duration_days} day(s)")
    print("-" * 60)
    diagnoses = diagnose(symptoms, fever_temp, travel_history, duration_days)
    if diagnoses:
        print("\n  DIFFERENTIAL DIAGNOSIS (Ranked by Confidence):\n")
        diagnoses.sort(key=lambda x: x[1], reverse=True)
```

```
        for i, (disease, confidence, risk, action) in enumerate(diagnoses, 1):
            print(f"  {i}. {disease}")
            print(f"     Confidence : {confidence}%")
            print(f"     Risk Level : {risk}")
            print(f"     Action     : {action}\n")
    else:
        print("  No matching diagnosis found. Consult a physician.")
    print("-" * 60)
    print("  DISCLAIMER: This is an educational decision support tool.")
    print("  Always seek professional medical advice for treatment.")
    print("=" * 60)

# ---- Test Cases ----
# Case 1: Malaria
run_diagnosis(
    patient_name="Rahul Sharma",
    symptoms=["chills", "shivering", "headache", "weakness"],
    fever_temp=39.2, travel_history="endemic_area", duration_days=4)

# Case 2: COVID-19
run_diagnosis(
    patient_name="Priya Mehta",
    symptoms=["dry_cough", "loss_of_taste", "fatigue"],
    fever_temp=38.0, travel_history="none", duration_days=3)

# Case 3: Influenza
run_diagnosis(
    patient_name="Amit Singh",
    symptoms=["dry_cough", "body_aches", "runny_nose", "fatigue"],
    fever_temp=38.9, travel_history="none", duration_days=2)
```

6. Results & Output

6.1 Test Case Summary

Three patient scenarios were constructed to exercise distinct diagnostic rules. The table below summarises the inputs and expected outputs for each case:

#	Patient	Symptoms	Temp (°C)	Travel	Expected Dx	Conf.
1	Rahul Sharma	chills, shivering, headache, weakness	39.2	Endemic area	Malaria	85%
2	Priya Mehta	dry_cough, loss_of_taste, fatigue	38.0	None	COVID-19	82%
3	Amit Singh	dry_cough, body_aches, runny_nose, fatigue	38.9	None	Influenza	90%

6.2 Program Output Screenshot

The screenshot below captures the actual terminal output produced by executing the Python program against all three test cases:

```
============================================================
   MEDICAL AI EXPERT SYSTEM - FEVER DIAGNOSIS REPORT
============================================================
  Patient Name   : Rahul Sharma
  Fever Temp     : 39.2 degC
  Symptoms       : chills, shivering, headache, weakness
  Travel History : endemic_area
  Illness Since  : 4 day(s)
------------------------------------------------------------

  DIFFERENTIAL DIAGNOSIS (Ranked by Confidence):

  1. Malaria
     Confidence : 85%
     Risk Level : High
     Action     : Urgent Blood Smear Test Required

------------------------------------------------------------
  DISCLAIMER: This is an educational decision support tool.
  Always seek professional medical advice for treatment.
============================================================

============================================================
   MEDICAL AI EXPERT SYSTEM - FEVER DIAGNOSIS REPORT
============================================================
  Patient Name   : Priya Mehta
  Fever Temp     : 38.0 degC
  Symptoms       : dry_cough, loss_of_taste, fatigue
  Travel History : none
  Illness Since  : 3 day(s)
------------------------------------------------------------

  DIFFERENTIAL DIAGNOSIS (Ranked by Confidence):

  1. COVID-19
     Confidence : 82%
     Risk Level : Medium-High
     Action     : RT-PCR test, isolation, monitor oxygen levels

------------------------------------------------------------
  DISCLAIMER: This is an educational decision support tool.
  Always seek professional medical advice for treatment.
============================================================

============================================================
   MEDICAL AI EXPERT SYSTEM - FEVER DIAGNOSIS REPORT
============================================================
  Patient Name   : Amit Singh
  Fever Temp     : 38.9 degC
  Symptoms       : dry_cough, body_aches, runny_nose, fatigue
  Travel History : none
  Illness Since  : 2 day(s)
------------------------------------------------------------

  DIFFERENTIAL DIAGNOSIS (Ranked by Confidence):

  1. Influenza (Flu)
     Confidence : 90%
     Risk Level : High
     Action     : Rest, fluids, antiviral medication

------------------------------------------------------------
  DISCLAIMER: This is an educational decision support tool.
  Always seek professional medical advice for treatment.
============================================================
```

Figure 1 — Terminal output of the Rule-Based Expert System for all three test cases

6.3 Observations

- Case 1 (Rahul Sharma, 39.2°C, endemic travel, chills + shivering): Rule R2 fired correctly, returning Malaria at 85% confidence with the action 'Urgent Blood Smear Test Required'.
- Case 2 (Priya Mehta, 38.0°C, dry cough + loss of taste): Rule R4 fired correctly, returning COVID-19 at 82% confidence with isolation and RT-PCR recommendations.
- Case 3 (Amit Singh, 38.9°C, dry cough + body aches): Rule R1 fired correctly, returning Influenza at 90% confidence with antiviral and rest recommendations.
- In all three cases, only the single most relevant rule fired—no spurious or incorrect rules were triggered—demonstrating clean, non-overlapping rule conditions for the given inputs.
- The sorting step in run_diagnosis() correctly ranks diagnoses from highest to lowest confidence when multiple rules fire simultaneously.

7. Discussion & Analysis

7.1 Strengths of the Approach

- Full Transparency: Every inference step is traceable to an explicit IF-THEN rule. Unlike black-box ML models, the system's reasoning can be fully audited by a clinician or developer.
- No External Dependencies: The use of standard Python ensures portability and reproducibility across all environments without dependency management overhead.
- Clinically Meaningful Output: By including confidence scores, risk levels, and specific recommended actions, the output mirrors the structure of a real clinical decision-support report.
- Differential Diagnosis: The system can return multiple ranked diagnoses from a single consultation, reflecting the reality of clinical practice where initial presentations are often ambiguous.

7.2 Limitations

- Static Rule Set: The knowledge base cannot adapt or improve with new data. Adding new diseases requires manual authoring of new rules.
- Binary Symptom Representation: Symptoms are encoded as present/absent flags. The system does not account for symptom severity, duration, or combinations that fall outside the predefined token set.
- No Uncertainty Propagation: Confidence scores are fixed per rule rather than derived from probabilistic inference (e.g., Bayesian networks), which could model uncertainty more rigorously.
- Scope Limitation: The system addresses only seven fever-related diseases. Clinical breadth would require a substantially larger and more complex knowledge base.
- No Lab Integration: Real diagnostic decisions often depend on laboratory investigations (CBC, malaria smear, RT-PCR). This system provides preliminary screening only.

7.3 Potential Extensions

- Fuzzy Logic: Replacing binary thresholds with fuzzy membership functions would allow the system to handle borderline temperature readings and partial symptom matches more gracefully.
- Bayesian Inference: Incorporating prior disease prevalence and conditional probability tables would provide statistically grounded confidence scores.

- Natural Language Input: Integrating an NLP pre-processing layer would allow free-text symptom descriptions to be parsed into the system's token vocabulary.
- Web/Mobile Interface: Wrapping the engine in a Flask or FastAPI backend with a mobile frontend would make the system accessible to patients and paramedics in field settings.

8. Conclusion

This experiment successfully demonstrated the design and implementation of a Rule-Based Expert System for medical diagnosis in the domain of fever-related diseases. The system encodes clinical knowledge as seven production rules and applies a Forward Chaining inference engine to patient data, producing ranked differential diagnoses with associated confidence scores, risk levels, and recommended clinical actions.

All three test cases executed correctly, with the appropriate diagnostic rules firing for each patient profile and producing clinically reasonable outputs. The experiment reinforces the foundational AI concepts of knowledge representation, inference mechanisms, and confidence-based reasoning, while situating them within a practical healthcare application.

While the system is intentionally limited in scope and serves as an educational decision-support tool rather than a clinical product, it illustrates the enduring relevance of expert systems in AI—particularly in contexts where transparency, auditability, and domain-specific precision are paramount.

9. Disclaimer

IMPORTANT: This Rule-Based Expert System is an educational prototype developed for academic study of Artificial Intelligence techniques. It is NOT a certified medical device, NOT approved for clinical use, and does NOT constitute medical advice. The diagnostic outputs are illustrative only and must not be used as a substitute for professional medical consultation, laboratory investigation, or clinical judgment by a qualified healthcare practitioner. Always seek the advice of a licensed physician for any medical condition.

References

Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Shortliffe, E. H. (1976). Computer-Based Medical Consultations: MYCIN. Elsevier.

Jackson, P. (1998). Introduction to Expert Systems (3rd ed.). Addison-Wesley.

Luger, G. F. (2009). Artificial Intelligence: Structures and Strategies for Complex Problem Solving (6th ed.). Pearson.

Python Software Foundation. (2024). Python 3 Documentation. https://docs.python.org/3/